

Networking

The PCAP sent to you was extracted from a video device in which communication we wanted to explore.

- Is there a video file in the capture?
A: Yes, there is a video
- What is the link for the video file which was requested?
A: <http://cdn.megaplay.xyz:80/live/jLn5maCdCk/ndc9zApfgx/404.ts>
- Can you replay the video from the packets captured?
A: Yes, 38 seconds extracted
- What is the content of the video? Is it a movie or a cartoon? What is the title of the video?
A: It's a Movie called "The Sorcerer's Apprentice" on HBO family
- What is the source IP address of the video server?
IP: 209.133.213.14
- What is the destination IP address of the client?
IP: 52.85.133.253
- Please share the video extracted.

Note: The video extracted is in a separate file called 'Video.ts'

Linux – part I

The pictures below show a troubleshooting question made by one of our engineers while trying to perform a forensic capture on a device. He wanted to copy the contents of a set of folders using the command RSYNC. When he tries to issue the script, it rapidly fills the destination space /dev/sdb1, and the computer stops working.

Can you identify the errors? What is he doing wrong?

Script content:

```
echo "Backing up important folders"
sleep 2
rsync -avzh /home arquivos/
rsync -avzh /etc arquivos/
rsync -avzh /usr/local arquivos/
rsync -avzh /usr/etc arquivos/
rsync -avzh /usr/local/src arquivos/
rsync -avzh /opt arquivos/
rsync -avzh /root arquivos/
rsync -avzh /var/lib arquivos/
rsync -avzh /var/mail arquivos/
rsync -avzh /var/www arquivos/
rsync -avzh /var/backups arquivos/
rsync -avzh /var/logs arquivos/
```

Output of command DF -h

```
Device      Boot Start        End  Sectors  Size Id Type
/dev/sdb1           32 15293439 15293408   7.3G  b W95 FAT32

root@kali:~#
root@kali:~#
root@kali:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0   1.9G   0% /dev
tmpfs           374M    12M   363M   3% /run
/dev/sda2       912G   8.6G   857G   1% /
tmpfs           1.9G    38M   1.8G   3% /dev/shm
tmpfs           5.0M     0   5.0M   0% /run/lock
tmpfs           1.9G     0   1.9G   0% /sys/fs/cgroup
tmpfs           511M   132K   511M   1% /boot/efi
/dev/sda1       374M    12K   374M   1% /run/user/130
tmpfs           374M    68K   374M   1% /run/user/0
tmpfs           7.3G   7.3G   1.4M 100% /root/test_pendrive
root@kali:~#
```

Answer:

The script uses a **relative path** for the destination: `arquivos/`.

- **Error:** If the engineer executed the script while inside the mount point of the pendrive (`cd /root/test_pendrive`), the destination folder `arquivos/` was created **inside** the pendrive. Since the script commands `rsync` to copy the **/root** directory, and the pendrive is mounted at `/root/test_pendrive`, the `rsync` command starts copying the pendrive into itself. It creates a **loop** of data:
`/root/test_pendrive/arquivos/root/test_pendrive/arquivos/root/...`
constantly replicating until every single byte of the 7.3GB is consumed.
- The reason the computer stopped working is due to **I/O Wait** saturation: The engineer used the **-z (compression)** flag. Compressing thousands of small files from `/etc` and `/var` while writing to a slow USB 2.0/3.0 bus creates a massive bottleneck. When the pendrive hit 100% capacity during this recursive loop, the **Linux Kernel** entered an uninterruptible sleep state while trying to manage the **cache** that had nowhere to go. This "locks" the CPU, causing the interface to freeze.

Linux – part II

Your team deploys a Python script that must run every 5 minutes to collect diagnostic data from multiple STBs.

You create two files:

/etc/systemd/system/stb-collector.service

```
[Unit]
Description=STB Collector Service

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/stb/collector.py
```

/etc/systemd/system/stb-collector.timer

```
[Unit]
Description=Run STB Collector every 5 minutes

[Timer]
OnCalendar=*:0/5

[Install]
WantedBy=timers.target
```

After enabling both with `systemctl enable --now stb-collector.timer`, the team reports:

- The timer **shows as active**, but the script **never runs**.
- `systemctl status stb-collector.service` shows:
- Failed to start STB Collector Service: No such file or directory
- `journalctl -u stb-collector.service` logs:
- `/usr/bin/python3: bad interpreter: No such file or directory`

Question:

1. Identify **all possible misconfigurations** in the service and timer files (there is more than one).

Identified Misconfigurations

.service file:

Incorrect Python Path (Main Error):

```
ExecStart=/usr/bin/python3 /opt/stb/collector.py
```

Systemd attempts to use the Python executable located at `/usr/bin/python3`. Since the log shows `bad interpreter: No such file or directory`, this file does not exist on this specific system. The script path `/opt/stb/collector.py` itself is correct, but the interpreter path is not.

Missing [Install] Section: Without this, `systemctl enable` does not know at which point during the boot process to register the service.

Missing User= : Without this, the service runs as `root`, which is unnecessary and poses a security risk.

Missing WorkingDirectory= : Without this, the script's working directory is unpredictable.

Missing Restart= : If the script fails, systemd will not attempt to recover it automatically.

In the .timer file:

Missing explicit Unit=:

Unit=stb-collector.service

Systemd infers which service to trigger by the filename it could work, **but it is implicit and fragile.**

2. Explain **why the timer appears active** even though the service never runs.
The timer and the service are **independent units**. The timer has only the responsibility to **count the time and trigger**.
It does not know if the service succeeded or failed.

The workflow may be something like this:

Timer counts 5 min (triggers the service)

Service attempts to start (fails)

terminates Timer: start another trigger in 5 minutes (unaware of the failure)

This is why the timer shows as **active** while the service shows as **failed**. They are two separate states.

3. Provide a **corrected version** of both files.

stb-collector.service

[Unit]

Description=STB Collector Service

After=network.target

[Service]

Type=simple

User=stb-collector

WorkingDirectory=/opt/stb

ExecStart=/opt/stb/python3 /opt/stb/collector.py

Assuming the `which python3` command was executed and the actual path was displayed.

Restart=on-failure

RestartSec=10s

StandardOutput=journal

StandardError=journal

[Install]

WantedBy=multi-user.target

stb-collector.timer

[Unit]

Description=Run STB Collector every 5 minutes

[Timer]

OnCalendar=:0/5*

Unit=stb-collector.service

AccuracySec=1s

Persistent=true

[Install]

WantedBy=timers.target

4. Describe a **complete troubleshooting workflow**, including at least these commands:

- `systemctl list-timers`
- `systemctl cat`
- `systemd-analyze verify`
- `journalctl -xeu`

WorkFlow:

Step 1 — Verify the state of timers

```
systemctl list-timers --all
```

Lists all timers, showing the last trigger and the next scheduled one. If `stb-collector.timer` isn't listed, it wasn't enabled. If it appears but "last trigger" never changes, the timer is firing but something is blocking execution.

Step 2 — View the exact content systemd is reading

```
systemctl cat stb-collector.service
systemctl cat stb-collector.timer
```

Shows the file as systemd sees it—useful if you edited the file but forgot to reload, as systemd might still be using the old version.

Step 3 — Validate file syntax

```
systemd-analyze verify /etc/systemd/system/stb-collector.service
systemd-analyze verify /etc/systemd/system/stb-collector.timer
```

It points out syntax errors and invalid configurations without needing to activate anything.

Step 4: Read error logs

```
journalctl -xeu stb-collector.service
```

Flags:

- x: Adds contextual explanations to errors.
- e: most recent events.
- u: Filters only logs for this specific service (This confirms the bad interpreter error and the incorrect Python path)

Step 5: Find the actual Python path

```
which python3
```

Returns the real path (e.g. `/opt/stb/python3`). This is the value that should go into `ExecStart=`

Step 6 — Manually test the service

```
systemctl daemon-reload
systemctl start stb-collector.service
systemctl status stb-collector.service
```

Test the service directly after fixing the files, without waiting for the timer.

5. Suggest **best practices** for deploying systemd services in production (paths, permissions, virtualenvs, logging, Restart policies).

Paths - Never Assume: Always verify where Python is located in the target environment using `which python3` before writing the `ExecStart`.

Permissions - Never Run as Root: Create a system user with no shell access (`/sbin/nologin`) and give them ownership of the script directory.

Logging - Centralize in the Journal: Use `StandardOutput=journal` so all prints and errors appear in `journalctl` with timestamps.

Restart - Automatic Recovery: Use `Restart=on-failure` with a `RestartSec` delay.

Reload After Changes: Always run `systemctl daemon-reload` after editing any unit file.

Development – part I

You are tasked with the development of a software which allows its users to send automated remote-control commands to set-top-boxes (STB). The system should include the following features:

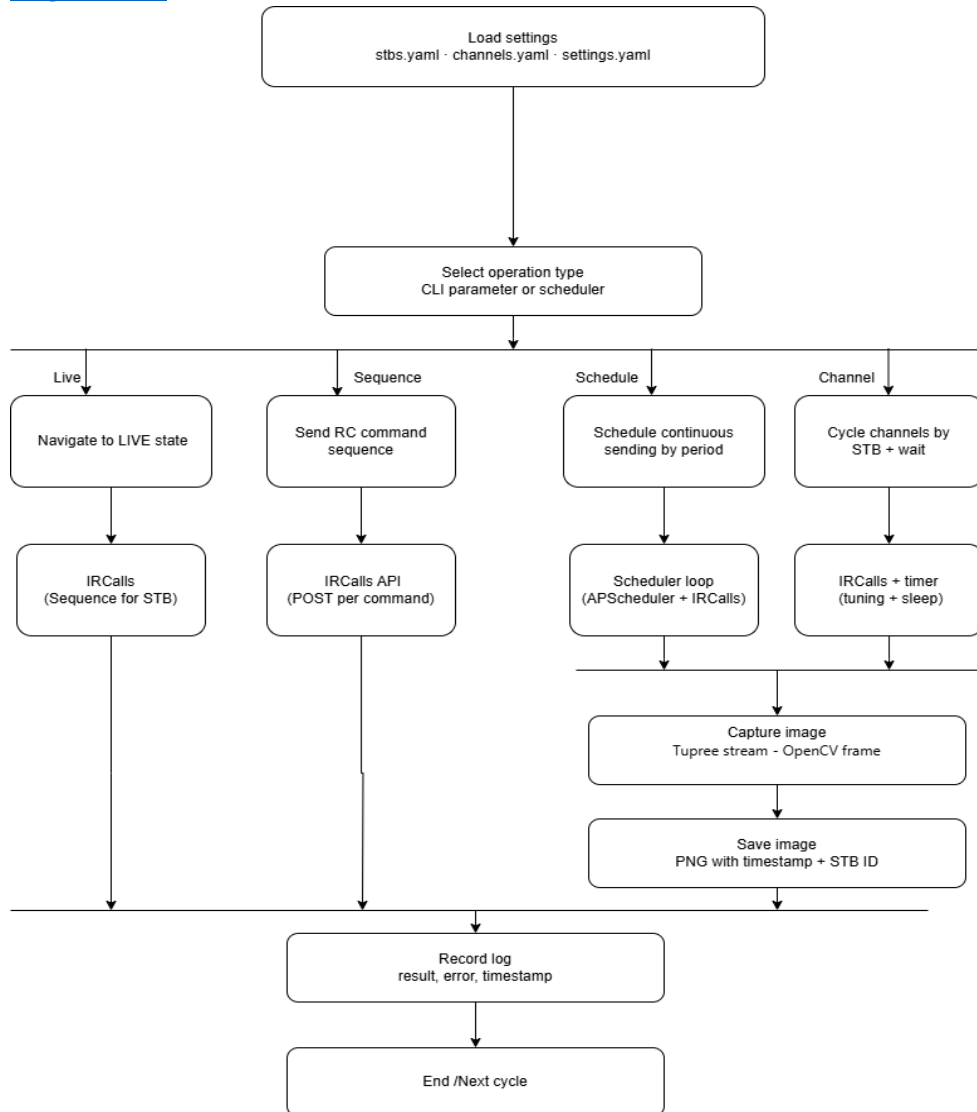
1. Send remote control command sequences to multiple different STBs.
2. Send specific remote control command sequences to the STBs to reach the LIVEⁱ state.
 - The STBs have different sequencies to reach the LIVE state:
For example:
 - STB1 will send commands: Right + Right + Left + OK
 - STB2 will send commands: Right + Down + Left + OK
 - STB3 will send commands: Right + Down + Down + Left + OK + OK
 - Your application needs to be ready to handle different sequencies
3. Send an indefinite amount of the same command during a set period to a set of STBs.
 - Your application must be able to “schedule” the commands.
4. Send different commands to specific STBs in a set and cycle through them (i.e., tune to specific channels for each one of the STBs).
 - Read a list of channels to tune
 - Your application must be able to read a list of channels like:
 - channel1, channel2 and channel3
 - And wait on desired channels during a period defined on configuration file:
 - Tune to channel1 and wait 60 seconds, then tune channel2 and wait 60 seconds, etc...
5. Capture images from the STB after sending the commands mentioned in items 3. and 4.

To achieve this, you have at your disposal the following infrastructure:

1. Linux and windows servers/containers able to connect to the rest of the components listed below except from item number 4. These reside in a network isolated from the STB. The software should be stored and executed in any of these.
2. Device able to send remote control commands to different outputs. You should communicate with this device through an API which receives simple web requests. This device holds the mapping for the different remote controls that interact with the STBs. For reference in pseudo code call this “IRCalls”.
3. Device which allows you to retrieve a live video feed from the STB. You communicate to this device through a .flv or .ts continuous stream which is available and visible in the internal network. (i.e., <http://192.168.5.101/0.ts>). For reference in pseudo code call it Tupree.
4. X number of different STB which can receive the remote-control commands from the software.

Based on what was stated above:

1. Share a pseudo-code for this application or a diagram that represents the workflow; you can use diagrams.net to draw it.



2. We would like you to use Python to develop this. What libraries for that programming language would you use?

requests - Call APIs and HTTP

opencv-python - Capture frames from streams

Pillow - Process and save images

APScheduler - Schedule commands

PyYAML - Read configuration files

threading/concurrent.futures - Parallel command delivery to multiple STB TVs

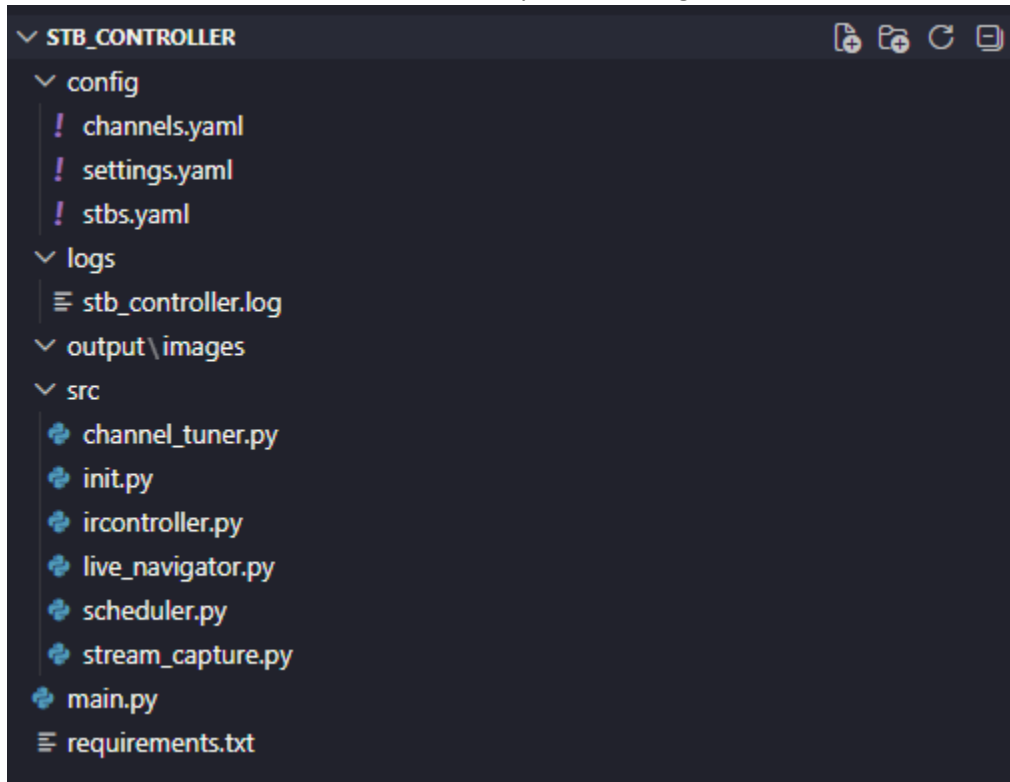
argparse - Command line interface

logging - Record events

time/datetime - Time control

OS/pathlib - Call system commands and handle paths/directories

3. Give us an idea of the structure of the files you'll be using.



4. How would you write the pseudo code that implements the features?

Navigation to the LIVE state

```
FUNCTION navigate_to_live(stb_id):
    SEQUENCE = CONFIG['stbs'][stb_id]['live_sequence']
    SUCCESS = TRUE
    FOR EACH command IN SEQUENCE:
        RESPONSE = ir_controller.send_command(stb_id, command)
    IF RESPONSE.status != 200:
        LOG_ERROR("Failed to send " + command + " to " +
            stb_id)
        SUCCESS = FALSE BREAK
    WAIT(500ms)
    IF SUCCESS:
        LOG_INFO("STB " + stb_id + " reached LIVE state.")
    ELSE:
        LOG_WARNING("LIVE navigation failed for " + stb_id)
```

Channel Cycling and Image Capture

```
FUNCTION cycle_channels(stb_id):
```

```

CHANNEL_LIST = CONFIG['channels']['list']

FOR EACH channel IN CHANNEL_LIST:
    COMMAND_SEQUENCE = channel.command_sequence

FOR EACH command IN COMMAND_SEQUENCE:
    ir_controller.send_command(stb_id, command)
    WAIT(500ms)
SLEEP(channel.wait_seconds)
FRAME = stream_capture.get_frame(stb_id)
IF FRAME IS NOT NULL:
    FILENAME = "output/images/" + stbid + "" + timestamp() +
    ".png"
    SAVE_IMAGE(FRAME, FILENAME)
    LOG_INFO("Captured channel " + channel.name + " for " +
    stb_id)
ELSE:
    LOG_WARNING("Stream unavailable for STB " + stb_id)

```

Scheduled Continuous Commands

```

FUNCTION schedule_periodic_command(stb_id, command,
interval, duration):
    END_TIME = current_time() + duration
TASK = FUNCTION():
IF current_time() < END_TIME:
    ir_controller.send_command(stb_id, command)
ELSE:
    SCHEDULER.removejob('job' + stb_id)
    stream_capture.capture_image(stb_id)
    LOG_INFO("Finished scheduled command job for " +
    stb_id)
    SCHEDULER.addjob(TASK, 'interval', seconds=interval,
id='job' + stb_id)

```

5. Any additional details are welcome.

- Keep all STB-specific data in configuration files, so new boxes, LIVE sequences, channels, or stream URLs can be added without changing the code.
- Add basic error handling for failed API calls or unavailable video streams, and register these events in the logs.
- Save captured images with the STB identifier and timestamp to make troubleshooting easier.

Regex

Based on file correlations.xlsx we need you to write REGEX on column H which would allow us to recognize the URLs corresponding on column C but also similar ones from the same providers.

Try to have the REGEX in column H to be loose enough to fit the text pattern in column C for that row, but not so loose as to match other rows.

Example:

- The URL from line 2, 3 and 4 have the “type content” (column F) and similar text pattern
- The URL from line 5 is from the same provider and has similar text pattern, but the “type” is “update” (column F)
- The first REGEX must match the lines 2, 3 and 4 but cannot match line 5 because it is another “type”, you can use the <https://regex101.com/> to test your REGEXs

Note:

The answer to this question is in a separate file called ‘correlations_answered.xlsx’ with all Regex included

Development – part II

Based on file correlations.xlsx, create and share a script in Python where we can extract the contents from column B and F and write in a txt file

The txt file must have unique entries avoiding any duplicate lines.

Wrong:

```
cdn2.fyre19.xyz, content
cdn2.fyre19.xyz, content
cdn2.fyre19.xyz, update
...
```

Correct:

```
cdn2.fyre19.xyz, content
cdn2.fyre19.xyz, update
```

File:

```
import pandas as pd
INPUT_FILE = "correlations.xlsx"
OUTPUT_FILE = "output.txt"

df = pd.read_excel(INPUT_FILE)

pairs = df[["SITE", "TYPE"]].dropna()
# Deduplicate
unique_pairs = pairs.drop_duplicates()

with open(OUTPUT_FILE, "w") as f:
    for _, row in unique_pairs.iterrows():
        f.write(f"{row['SITE']}, {row['TYPE']}\n")
print(f"Done! {len(unique_pairs)} unique entries written to
{OUTPUT_FILE}")
```

Output:

```
cdn2.fyrel9.xyz, content
cdn2.fyrel9.xyz, update
user.telebox.club, auth
www.potisata.com, update
www.potisata.com, system
saepg.mi845mk4.com, ep
sa.mi845mk4.com, system
sa.mi845mk4.com, ep
upgrade.tvtvapk.com, content
upgrade.aecattv.com, content
update.aecattv.com, update
aiauth.aiiptv.com, auth
```

Docker

An engineer builds the following Dockerfile to create an API service:

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt

EXPOSE 8080
CMD ["python", "api.py"]
```

The container builds successfully and starts with:

```
docker run -d --name api -p 8080:8080 myapi:latest
```

However, the API is **not reachable** on the host at `http://localhost:8080`.

Additional symptoms:

- docker logs api shows the app running on
Listening on 0.0.0.0:5000
- Changing code requires rebuilding the full image each time, which takes **too long**.
- A second engineer complains that the container is **1.2GB** even though the base image is lightweight.
- Running `docker exec -it api sh` shows the `/app` folder contains many files that should have been `.dockerignore-excluded` (logs, git folder, test data).

Question:

1. Identify **ALL issues** related to:
 - port configuration
 - incorrect host binding
 - oversized image due to missing `.dockerignore`
 - inefficient layering strategy
 - rebuild times

1. Issue Identification

Port Configuration & Host Binding: The application is listening on port **5000** (as shown in logs), but the `docker run` command and `EXPOSE` instruction use **8080**. The host cannot reach the service because of this mismatch.

Oversized Image: The absence of a `.dockerignore` file causes the `COPY . .` command to pull in unnecessary bulk like `.git`, `__pycache__`, logs, and local virtual environments, inflating the image to 1.2GB.

Inefficient Layering: Placing `COPY . .` before `RUN pip install` breaks Docker's layer caching. Any minor code change forces a complete re-installation of all dependencies, drastically increasing rebuild times.

2. Provide a **corrected Dockerfile** using:

- o multistage builds
- o proper `.dockerignore`
- o explicit port configuration

Corrected

Docker:

```
FROM python:3.11-slim AS builder WORKDIR /app COPY
requirements.txt .
RUN pip install --no-cache-dir --user -r
requirements.txt
FROM python:3.11-slim AS runner WORKDIR /app

ENV PYTHONUNBUFFERED=1 ENV PATH=/root/.local/bin:$PATH

COPY --from=builder /root/.local /root/.local
COPY . .
RUN useradd -m appuser USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s
CMD curl -f http://localhost:5000/health || exit 1
CMD ["python", "api.py"]
```

.dockerignore FILE:

```
.git
pycache/
*.pyc venv/
.env
logs/
tests/
.vscode/
Dockerfile
docker-compose.yml
```

3. Explain how to diagnose portbinding problems using:
 - `docker ps`
 - `docker port`
 - `ss -tlnp` or `netstat` inside the container
- **docker ps**: Check the PORTS column. It should show the mapping, e.g., `0.0.0.0:8080->5000/tcp`.
- **docker port api**: Directly displays which host port is bound to the container's internal port.
- **Inside the container (ss -tlnp / netstat)**: Run these via `docker exec`. If the output shows `127.0.0.1:5000`, the app is only reachable internally. It must show `0.0.0.0:5000` or `:::5000` to accept external traffic.
4. Suggest improvements for **local development** (e.g., docker compose, bindmount volumes, watch mode).
 - **Docker Compose**: Orchestrate the API and dependencies (like databases) in a single file.
 - **Bind Mounts**: Use `-v $(pwd):/app` in your compose file to sync local code changes instantly without rebuilding.
 - **Watch Mode**: Use frameworks with "hot reload" (like FastAPI or Flask debug mode) so the server restarts automatically upon file saves.
5. Suggest improvements for **production** (e.g., nonroot user, minimal base image, healthchecks).
 - **Non-root User**: Minimizes security risks if the application is compromised.
 - **Minimal Base Images**: Stick to `-slim` or `-alpine` versions to reduce the attack surface.
 - **Healthchecks**: Allows orchestrators (like Kubernetes or Docker Swarm) to automatically restart containers if the service hangs.

English

We will provide you with the executive summary and key sections of the report: **ONLINE COPYRIGHT**

INFRINGEMENT IN THE EUROPEAN UNION (2017-2023). Please focus your analysis specifically on **Section 3 (IPTV Piracy)**, **Section 4 (Data and Methodology)**, and **Section 5 (Evolution of Piracy by Content Type)**. Please provide a summary (no more than one page) in English covering:

1. **The IPTV Business Model:** How it mimics legitimate services and why it represents a "professionalized" threat compared to old-school piracy.
2. **Socio-Economic Drivers:** Based on the data in Section 4, explain how factors like GDP per capita and income inequality influence piracy rates across different EU Member States.
3. **Content Evolution:** Identify which content types (e.g., TV, Live Sports, Software) have seen recent growth and explain why this shift is occurring.
4. **Technical Solutions:** Propose three **modern technical means** (relevant to 2026) to mitigate the specific "IPTV and Live Event" piracy described in these sections.

Note: The written response to this question is in a separate file called 'piracy report analysis.pdf'

ⁱ LIVE state: The state in which a STB is reproducing LIVE audio/video as an end-user would be consuming it his/her house.